5.12 Spirograph (Lab 5.2)

LAB ACTIVITY | 5.12.1: Spirograph (Lab 5.2) | Full screen | 0 / 60

In this exercise, you will use the turtle graphics package to generate pictures like those which can be made with a spirograph set. A Spirograph is formed by rolling a circle inside or outside of another circle. The pen is placed at any point on the rolling circle. See http://en.wikipedia.org/wiki/Spirograph.

Download this turtle-based template code and rename it `spirograph.py`.

- template-turtle.py

We will use iterative development to develop a program that allows a user to generate spirograph pictures.

Iteration 1

Implement the following algorithm:

1. ***Import*** *the turtle and math libraries.* (These first two lines are already included in the template.)
2. *Create the turtle window and a turtle.*
3. **Set** `moving_radius` equal to a floating point value read from the user for the radius of the moving circle.
4. **Set** `fixed_radius` equal to a floating point value read from the user for radius of the fixed circle.
5. **Set** `pen_offset` equal to a floating point value read from the user for the offset of the pen point in the moving circle.
6. **Set** `color` equal to a string value for the color of the pen.
7. *Draw the spirograph.* (Save the implementation of this step for a later iteration.)

Implement this non-function part of the algorithm before proceeding. Here is a sample execution, with the required prompts and no other output.

```
moving radius: 55.0↵
fixed radius: 53.0↵
pen offset: 77.0↵
color: gold↵
```

Iteration 2

A spirograph picture can be drawn by using these parametric equations:

$$x(t) = (R + r) \cdot \cos(t) + p \cdot \cos\left(\frac{(R + r) \cdot t}{r}\right)$$

$$y(t) = (R + r) \cdot \sin(t) + p \cdot \sin\left(\frac{(R + r) \cdot t}{r}\right)$$

In these equations:

- *t* is the current "time".
- *R* is the radius of the fixed circle.
- *r* is the radius of the moving circle.
- *p* is the offset of the pen point in the moving circle.

To draw a spirograph, you will need to do two things, both of which will be coded in the space you left in the algorithm specified above. First, you need to get set up and then you need to use a loop to move the turtle over "time". Do this as follows.

1. ***Import*** *the turtle and math libraries.*
2. *Create the turtle window and a turtle.*

3. ***Set*** *`moving_radius` (r in the equations) equal to a floating point value read from the user for the radius of the moving circle.*
4. ***Set*** *`fixed_radius` (R in the equations) equal to a floating point value read from the user for radius of the fixed circle.*
5. ***Set*** *`pen_offset` (p in the equations) equal to a floating point value read from the user for the offset of the pen point in the moving circle.*
6. ***Set*** *`color` equal to a string value for the color of the pen.*

   # First, do the spirograph setup.
7. Tell your turtle to pick up its pen.
8. Tell your turtle to move to the appropriate initial position, i.e., use the equations given above to compute x(0) and y(0).
9. Tell your turtle to put the pen down.
10. Tell your turtle to set the pen color to the value given by the user.

    # Then, draw the spirograph.
11. **Set** `t` equal to 0.0.
12. Loop while the `t` is less than 360.
    a. Increment `t` by 0.1.
    b. Compute a value for `x` using the `t` as the value of *time*. Use the equations specified above.
    c. Compute a value for `y` using the `t` as the value of *time*.
    d. Tell your turtle to go to the location specified by `x` and `y`.

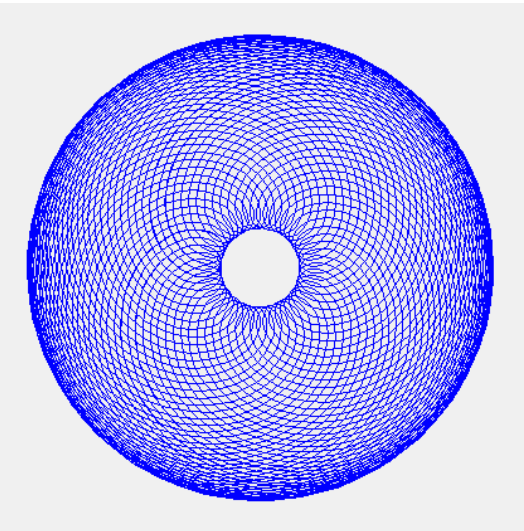
Note that drawing the full 360 degrees of the spirograph can take some time. For such complex drawings, it can be helpful to run:

- `pen.hideturtle()` — Hiding the turtle makes the drawing process look cleaner.
- `pen.speed(0)` — This tells the pen to run at full speed.
- `turtle.tracer(False)` - This turns off tracing entirely.

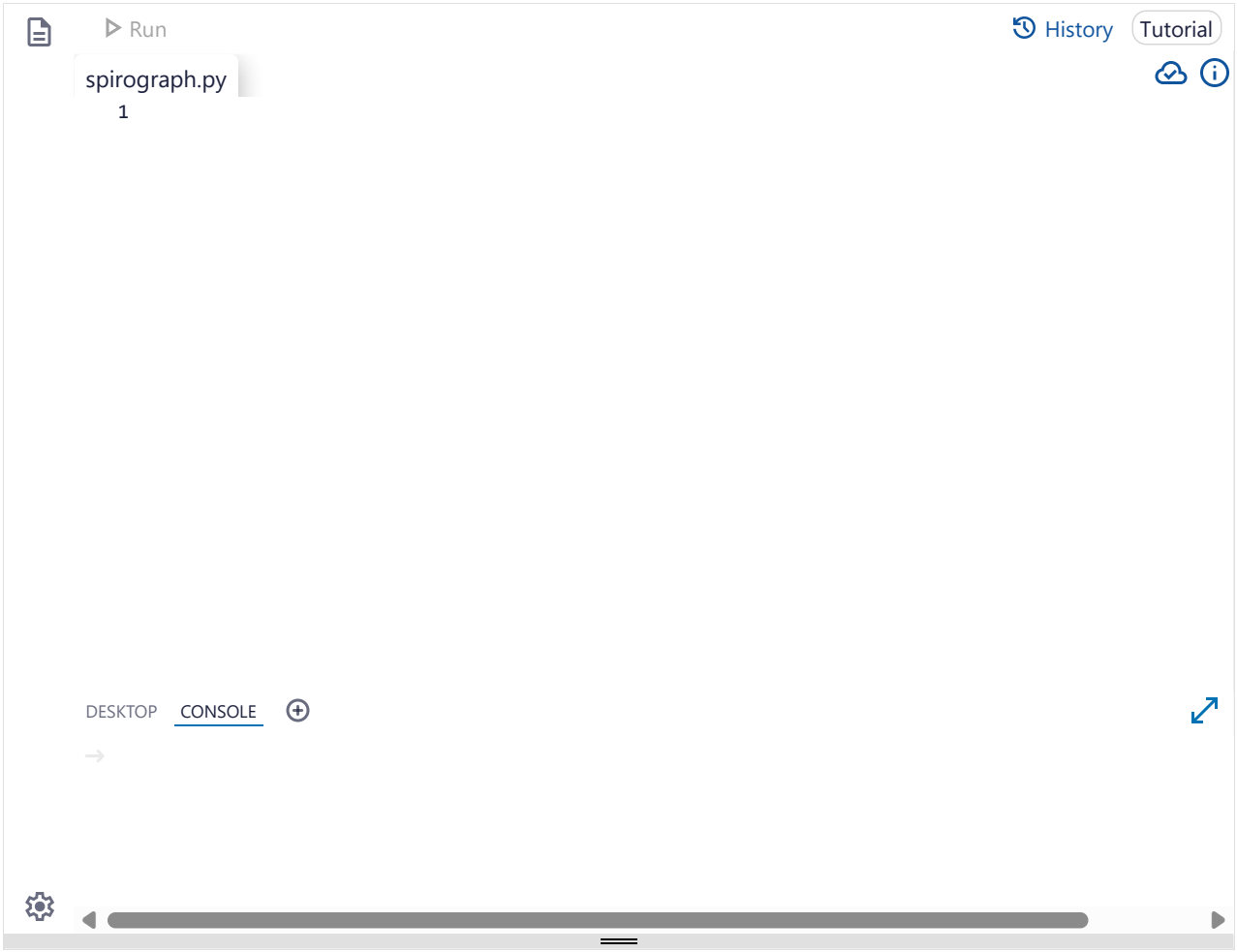
Assuming you have carefully followed the algorithm, you should be able to test your code now. For example, you might try these values:

- 55.0, 53.0, 77.0, gold
- 25.0, 23.0, 77.0, red
- 79.0, 74.0, 38.0, blue

Here is the output for the first example (55, 53, 77, 'gold').



Open new tab | Dock



Submit for grading

Coding trail of your work What is this?

```
History of your effort will appear here once you begin working on this
zyLab.
```

Latest submission

No submissions yet

Template, Model Solution, and Test Bench Show

Lab statistics Show

Student behavior insights (beta) Show

Similarity detection Show

Student activity Show